🚦 Performance | Use-As-Dictionary

Project	Use-As-Dictionary
Brands	Cross brand
Points of Contact	
Performance	Web Performance Team (Giorgio Pellegrino Christian Silvestri Daniele Merola Domenico Cavaglieri Lorenzo Bartomioli emanuele.galeotti)
References	Supercharge compression efficiency with shared dictionaries | Blog | Chrome for Developers

Introduction
1. Build-Time Dictionary Generation
2. Head HTML manipultion
3. CDN Configuration
   Use-As-Dictionary
   Content-Encoding
   Vary
   Dictionary caching

Introduction

The objective of this document is to define an implementation approach within the current Luxottica architecture for serving resources using **dcb** content encoding.

> The objective of this activity is to compress resources using a dictionary-based compression system, achieving compression rates of up to 90%.

The implementation requires changes across multiple layers of the architecture, including the application build process, head HTML manipultion, and CDN configuration.

1. Build-Time Dictionary Generation

The first step is to configure the application build process, to generate a dictionary file for the resources (in this documentation one CSS) and the resources compressed.

The main objective is to generate the dictionaries during each frontend build whenever they are missing or the resource compression requirements are no longer met. The latter check is particularly important to ensure long-term maintainability, as resources may change over time, potentially requiring the dictionaries to be regenerated accordingly.

To integrate this step into the build process, please include the following plugin in the source code. You can download it here:



The plugin can be configured in `webpack.config.js` as follows:

```
const raybanCssInput = path.resolve(__dirname, 'src/styles.[personal-hash].css');
    new CompressionDictionaryPluginProd({
      input: raybanCssInput,
      dictionarySource: 'dictionaries/dictionary_styles.dat',
      autoUpdate: {
        minSavings: 0.9,
      },
      dcbFilename: `dist/${path.basename(raybanCssInput)}.dcb`,
    }),
```

Once the build is complete, the following resources will be generated in the `build` folder:

```
dictionary_styles_[hashSHA-256].dat
```

The hash is calculated using SHA-256 algoritm to identify in a unique way the resource

```
styles.[personal-hash].css
```

```
styles.[personal-hash].css.dcb
```

The resource compressed maintained the same name of the original resource.

```
compression-dictionary-manifest.json
```

The manifest contains the Base64-encoded hash associated with the URL of each compressed resource. It can be deployed on the same server as the website or stored in an alternative location, such as an FTP server, Welcome to EdgeKV, or User-defined variables. This approach allows the manifest to be retrieved dynamically whenever required.

2. Head HTML manipultion

The HTML document should instruct the browser to download and cache the dictionary using the standardized compression-dictionary link relation.

The syntax is:

```
<link rel="compression-dictionary" href="/dictionary_style_[hashSHA-256].dat">
```

The `rel="compression-dictionary"` link must be injected server-side by reading the URLs specified in the `compression-dictionary-manifest.json` file.

The browser can download this dictionary during an appropriate idle period and subsequently use it when requesting matching resources.

The CSS resource can be referenced normally:

```
<link rel="stylesheet" href="/styles.[personal-hash].css">
```

3. CDN Configuration

The last step is to configure the response headers for the resources that will be eligible for dictionary-based compression.

Use-As-Dictionary

The dictionary resource must return a `Use-As-Dictionary` response header defining which resources can use it as a compression dictionary.

For example:

```
Use-As-Dictionary: match="/dist/styles.*.css"
```

The `match` directive supports URL patterns and wildcards, allowing the same dictionary to be used for multiple versioned resources.

Content-Encoding

The CSS resource compressed using the shared Brotli dictionary must be served with:

```
Content-Encoding: dcb
```

⚠ **Only if** the browser sends the following request headers:

```
Accept-Encoding: gzip, br, zstd, dcb, dcz
Available-Dictionary: :<base64-sha256-hash>:
```

the CDN should verify that the corresponding dictionary is listed in the manifest resource uploaded during the deployment phase.

If the dictionary is valid and available online, the CDN should serve the corresponding compressed resource by appending the `.dcb` extension to the original resource path.

For example:

`/styles.[personal-hash].css` → `/styles.[personal-hash].css.dcb`

This behavior can be implemented using Akamai Property Manager's **Modify Outgoing Request Path** functionality. Please refer to the following documentation for further details: Modify Outgoing Request Path .

Vary

Because the response varies depending on whether the client supports `dcb` and which dictionary is available, the compressed resource should also include:

```
Vary: Accept-Encoding, Available-Dictionary
```

Dictionary caching

The dictionary resource should be cached by the browser for at least two weeks (average time of luxottica releases).

The recommended response header is therefore:

```
Cache-Control: public, max-age=1209600, immutable
```